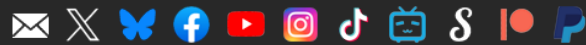


Inigo Quilez :: articles :: stereo - 2016



Intro

This article deals with fast stereo rendering. Stereo rendering is one of the basic building blocks of **VR rendering**, so it's worth spending some time trying to get it implemented right. One of the things novices mistakenly assume is that doing stereo is simply rendering the shadows and other eye-independent effects once and then rendering the rest of the scene twice. Or in other words, some variation of calling the render function in their game loop twice with different 'cameras'. This is a natural thing to do, but as we are going to see in this article, this is not how you do performant stereo rendering.

Basics

So, the idea here is to get efficient stereo rendering, or in other words, get the engine to render stereo in less than twice the time it takes to render mono. As mentioned above, the most basic and naive implementation of stereo rendering would be something like this:

```
DoProcessEye();           // Do frustum culling, compile draw command/index
DoRenderEyeCommon();      // Render eye-shared shadow maps, and distant objects
for( int i=0; i < 2; i++ ) // stereo
{
    DoRenderEye( i );      // Render regular geometry and postprocess effects
}
```

or written in more detail:

```
DoProcessEye();
DoRenderEyeCommon();
for( int i=0; i < 2; i++ )
{
    BindRenderTarget( i )
    SetViewport( i );
```

```

        SetEyeUniforms( i );

        for( int j=0; j < numObjects; j++ )
        {
            BindState( j );
            DrawObject( j );
        }
    }

layout (row_major, binding=1) uniform EyeUniforms
{
    mat4x4      mMatrix;
    mat4x4      mMatrixInverse;
};

```

This will produce a correct stereo render, but not in an efficient way. First, the whole collection of visible objects is traversed twice. This can be a CPU burden. Also, this code produces twice as many render calls as a mono render, which is no good. Lastly, state changes such as shaders, uniforms and textures (represented by BindState() in the code) are duplicated as well, which can be a huge bottleneck. Lastly, the GPU will have to perform vertex pulling, shading and tessellation twice as well, something that can become a bottleneck as well.

The central idea behind rendering stereo efficiently is to reverse the order of the inner and outer loops above, and transform the code into the following:

```

DoProcessEye();
DoRenderEyeCommon();

BindRenderTarget()
SetViewPorts();
SetEyeUniforms();

for( int j=0; j < numObjects; j++ )
{
    BindState( j );
    for( int i=0; i < 2; i++ )
    {
        DrawObject( i, j );
    }
}

layout (row_major, binding=1) uniform EyeUniforms
{
    struct Eye
    {
        mat4x4      mMatrix;

```

```

        mat4x4      mMatrixInverse;
    }mEye[2];

    mat4x4      mMatrix;
    mat4x4      mMatrixInverse;
};

```

Basically, instead of doing `foreach(eye)foreach(object)` we change to `foreach(object)foreach(eye)`. The benefits are clear even at first sight: we have reduced the state changes to half. For this to work, we need to be able to rendering the left and right eyes without switching render targets, which is an expensive operation. For this to be possible we'll need one single render target but it will be twice as big usual. Basically, the same render target/texture will hold both the left and right eye. This seems like a great waste of space in most cases, since the multisampled buffers now need to be 2x as big (of course, the post-resolve and post-color-grade/processing buffers stay the same, but those are cheap anyways since they are 1 sample per pixel anyways). Despite this extra 2x memory cost in all these expensive buffer, I am happy to pay given the performance benefits it enables.

The other thing to note is that now the eye information uniform actually contains both the left and right eye transformation matrices. This, together with the fact that we uploaded both eye's viewport descriptions (check `glViewportIndexed` to see how to achieve this in OpenGL), will allow us to project the geometry correctly as seen from the left or right eye and route it to the correct viewport in the big left+right compound render target. The per-eye transformation are in the `mEye[2]` array, and the global viewer position is right after those two. As explained in the [basic VR rendering article](#), use the formers for projection/rasterization purposes and the others for culling, level of detail, etc. Obviously, for this to work all you need to do is to index into the right `mEye` member based on the left/right index `i` within your shaders.

There's a few ways to do this. From less efficient to more efficient, some of the ways are:

1. Call `DrawIndexed()` twice, use a uniform to do projection+viewport routing (in the Vertex Shader)
2. Call `DrawIndexedInstanced()` once with instancing of 2 and use `gl_InstanceID` to do projection+viewport routing (in the Vertex Shader)
3. Call `DrawIndexed()` once with a Geometry Shader doing 2 invocations, and use the invocation id to do projection+viewport routing

The three methods do the viewport routing by using some mechanisms like `gl_ViewportIndex` (or `gl_LayerID` if you don't have multiviewport but have layered textures). For the sake of using modern and efficient APIs I am not considering the trivial case of doing it all the oldschool way with two `DrawIndexed()` with different projection and viewport state per call, nor doing manual viewport-ing with custom clipping planes. Now, the

differences between the three proposed methods are these:

- The first option is the less efficient of the three. Beware we are benefiting from all the savings in state changes, but we are still processing all the vertices of each mesh twice and doing 2 render calls per objects.
- With option number two, we are reducing the render calls to only one per instance. Routing is as simple as `gl_ViewportIndex = gl_InstanceID`. However, we are still running the vertex shader twice per object (meaning, we are reading from the vertex buffer memory twice and we are running the primitive assembly twice).
- Option number three performs only one render call per object as well and runs the vertex shader only once. The duplication and routing happens in a Geometry Shader by specifying the number of invocations to be 2. These two invocations happen in parallel, as far as I know. In OpenGL, you can achieve this by using the qualifier `"layout(invocations = 2) in;"`. Then, viewport routing is done with `gl_ViewportIndex = gl_InvocationID`.

My preferred method is 3, since I am using Geometry Shaders anyways. When I don't have one, I sometimes add one to enable the technique, and sometimes I use method 2 if the mesh is not too heavy. This works because `gl_ViewportIndex` can in fact be used in the Vertex Shader as well if your card has the `GL_ARB_shader_viewport_layer_array` extension. IN my case I got excellent performance of my invocation system in the Geometry Shader, method 3. Try each in your demo/game see what happens.

So, after all the rearchitecture, the final code looks like this:

```
DoProcessEye();
DoRenderEyeCommon();

BindRenderTarget()
SetViewPorts();
SetEyeUniforms();

for( int j=0; j < numObjects; j++ )
{
    BindState( j );
    DrawObject( j ); // either DrawIndexed() + GS
                    // or DrawIndexedInstanced() + VS
}

layout (row_major, binding=1) uniform EyeUniforms
{
    struct Eye
```

```
{  
    mat4x4      mMatrix;  
    mat4x4      mMatrixInverse;  
}mEye[2];  
  
mat4x4      mMatrix;  
mat4x4      mMatrixInverse;  
};
```

Conclusions

With a little bit of work it is possible to remove the state changes, render call and vertex processing bottlenecks when doing stereo rendering, leaving only the pixel shading and framebuffer bandwidth bottlenecks. Optimizing those is for another article.

[inigo quilez](#) - learning computer graphics since 1994